

Learn from examples. *Generate with fixed weights.*

Connect data and training to assistant behavior,
then trace how a checkpoint generates a reply.

🕒 20-25 min core reading · 20-30 min practice · 12 short sections

A base language model may continue a movie review. An instruction-tuned model is trained to answer a request about that movie. Both predict tokens, but their training examples and objectives shape different behavior. This chapter connects the training pipeline to the moment an assistant starts generating a reply.

WHAT YOU WILL UNDERSTAND

The main question: what changes while a model learns, and what changes while it answers?

The practical path: prepare text, predict targets, update parameters, adapt with demonstrations, then generate from a frozen checkpoint.

Reading path: twelve short sections, three diagrams, a training experiment, and a Python workbook. Optional notes hold the extra detail.

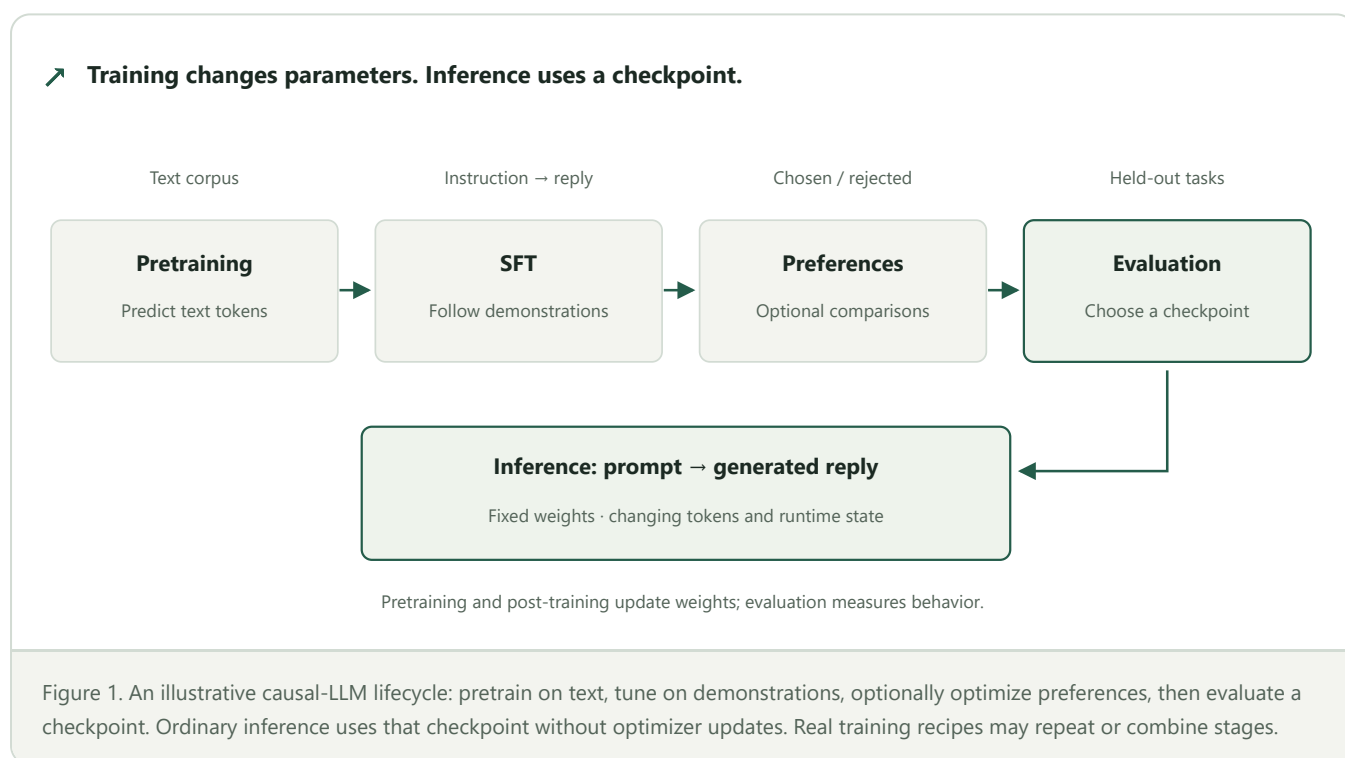
1. Separate training from using the model

Training adjusts learned parameters using an objective and an optimizer. **Inference** uses those parameters to calculate outputs. During an ordinary chat response, the prompt and generated sequence change; the model does not perform an optimizer update after every message.

Imagine a movie assistant. Training can teach it how recommendation conversations tend to work. At runtime, your request, "Suggest a gentle comedy," supplies context. Retrieved catalog entries supply additional evidence. Neither operation, by itself, changes the model's weights.

Stage	Learning signal	Intended change
Pretraining	Tokens from a large text collection	Learn broad predictive patterns
Supervised fine-tuning (SFT)	Demonstrations of desired replies	Adapt behavior to instructions and formats
Preference optimization	Comparisons or reward signals	Favor responses that meet chosen criteria
Inference	Current prompt and generated prefix	Produce an answer using a checkpoint

This is a useful lifecycle, not a mandatory recipe for every model. Post-training can combine or repeat methods, and evaluation belongs throughout the process.



2. Build the data before training the network

A text collection becomes a training dataset through deliberate choices: identify sources and permissions, preserve provenance, remove unsuitable or sensitive material, filter quality, reduce duplicates, choose a mixture, and tokenize it. For a movie corpus, reviews, subtitles, catalog descriptions, and conversations have different styles and biases.

Keep evaluation material separate at meaningful boundaries. Splitting nearly identical reviews across training and validation can make memorization look like generalization. A movie benchmark's answers appearing in pretraining also contaminate the measurement. Follow Chapter 12's split and leakage principles before tuning preprocessing on evaluation results.

Tokenized documents are divided or packed into sequences. Boundaries matter: an end marker identifies a document ending, but does not automatically block attention across packed examples. Packing, causal visibility, and loss masking must match the intended training design.

The model does not become an exact searchable archive. Weights capture statistical patterns and can also memorize material. Use retrieval when an application needs inspectable, current movie evidence; a fluent continuation is not evidence of a source lookup.

▼ Go deeper: data quantity, mixture, and repeated exposure

Count **tokens processed**, not just files downloaded. More tokens can mean new examples, repeated examples, or a different tokenizer. Repetition can increase an example's influence without adding information. Our workbook deliberately repeats short examples so that the weighting is visible; it is not a production data-cleaning policy.

Changing the mixture changes the objective being optimized. Mostly English reviews with few other languages will not measure or teach every language equally. Track source, language, topic, duplication, and date alongside aggregate loss. Maintain an untouched final test set, and use validation data for choices during development.

Compute, parameter count, and training-token count must be considered together. The Chinchilla study showed that allocating a fixed compute budget between model size and data can substantially change performance. Its findings are not a universal token-to-parameter prescription for every architecture, dataset, or deployment goal.

3. Train on the next token

For a causal language model, each prefix predicts the next token. Given `comedy is warm .`, inputs and targets are shifted: the position containing `is` predicts `warm`; the position containing `warm` predicts the period. The model produces vocabulary logits, softmax produces probabilities, and the correct observed token supplies the target.

Average negative log-likelihood over selected targets:

$$L(\theta) = -\frac{1}{N} \sum_{t \in S} \log p_{\theta}(x_t \mid x_{<t}), \quad N = |S|.$$

θ represents trainable parameters, S is the set of scored target positions, and $x_{<t}$ is the earlier prefix. A target probability of 0.25 contributes $-\ln(0.25) \approx 1.3863$ nats. Increasing that target's probability reduces its loss.

Teacher forcing uses the observed earlier tokens during training. A causal transformer can calculate many positions together because their inputs are already available; the attention mask still prevents access to each future target. The loss then flows backward through the network, and an optimizer updates parameters. This connects Chapters 20-24: forward pass → loss → backpropagation → update.

4. Track a training run and save checkpoints

A **batch** groups examples or tokens for an update. A **step** usually means one optimizer update; gradient accumulation can combine several smaller forward/backward batches into that update. An **epoch** is one pass through a defined dataset, although token budgets are often more useful for large streams of text.

Learning rate controls update size. Training loss shows fit to training targets; held-out loss helps detect overfitting. Checkpoints save progress so you can evaluate, compare, resume, or deploy a particular state. To resume training faithfully, weights alone are insufficient: optimizer state, scheduling, random state, and data position can also matter.

In the workbook, every update sees the entire tiny dataset. Pretraining always runs for 80 updates. You then choose 0, 20, or 80 SFT updates. These are teaching settings, not practical settings for training a large language model.

▼ Go deeper: parameters, activations, and memory budgets

Parameters are learned numbers shared across inputs. **Activations** are intermediate results produced for a particular batch. **Gradients** measure how a loss changes with parameters, and an optimizer may keep additional state such as running moment estimates. Training memory must account for all of these, not just a downloaded weight file.

Longer sequences and larger batches affect memory and computation. Mixed precision changes numerical representation; distributed training spreads work or stored state across devices. These methods preserve a training objective while changing how it is executed, subject to numerical and implementation differences. Chapter 26 will connect these costs to efficient LLM systems; you do not need a GPU cluster for the current workbook.

5. Teach the model to follow instructions

A base model learns to continue text. It may answer a question, repeat it, or continue an imagined conversation. **Instruction tuning** uses demonstrations of the behavior you want: a request paired with an appropriate response, often serialized as a conversation.

For our fictional movie catalog, a simplified demonstration is:

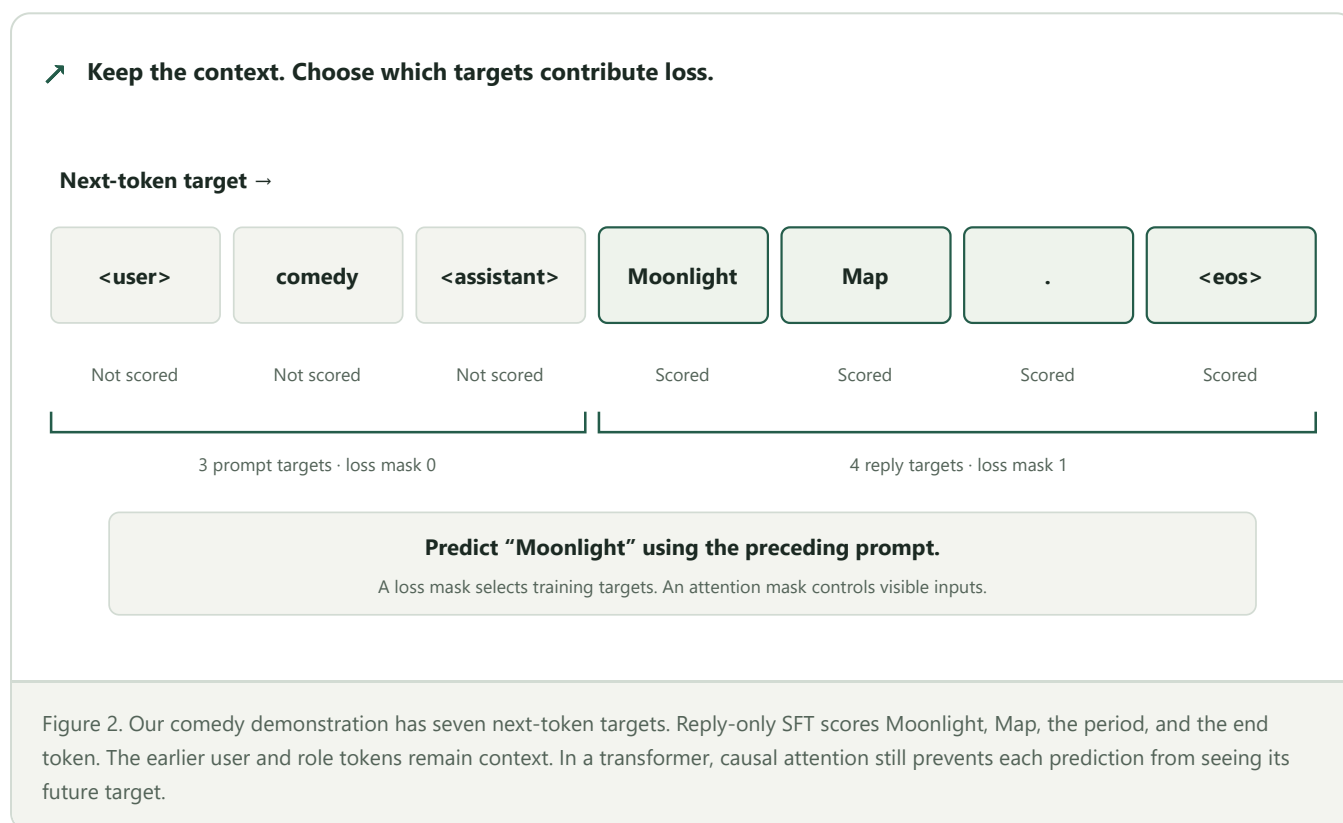
```
<user> comedy <assistant> Moonlight Map . <eos>
```

SFT still predicts tokens. The distinction is the training distribution and which targets are scored. Better demonstrations cover varied requests, formatting, missing information, and appropriate uncertainty; repeating one polished response does not teach every case. SFT can update all parameters or a smaller trainable subset, such as adapters. Chapter 27 develops fine-tuning choices.

Pretraining and SFT need not use different network architectures. Continuing to train on raw movie reviews is **continued pretraining**; training on request-reply demonstrations is SFT. Adding a review to the current prompt is neither of these.

6. Distinguish a loss mask from an attention mask

With **reply-only loss**, the prompt stays in the input but its targets do not contribute direct loss. The model learns to predict the assistant's response conditioned on that prompt. In our example, the three prompt targets are unscored; `Moonlight`, `Map`, `.`, and the end token are scored.



An **attention mask** controls which input positions a representation may use. A **loss mask** controls which predictions count toward the objective. Excluding user targets from loss does not mean hiding the user's request. The TRL SFT documentation describes assistant-only and completion-only losses; correct support also depends on the conversation format and template.

▼ Go deeper: gradients can still pass through the prompt

In a neural model, a reply prediction depends on representations of earlier prompt tokens. Its loss can therefore send gradients through computations involving those prompt positions, even when they have no directly scored targets. "No direct prompt loss" does not mean "no learning involving prompt representations."

Our miniature model is different: each preceding token pair selects an independent trainable row of logits. Reply-only SFT updates rows disjoint from its prose rows, so prose loss stays unchanged. A transformer shares parameters across contexts and can experience **catastrophic forgetting**: adaptation degrades an earlier capability. Evaluate retained capabilities as well as the new task. The workbook's isolated rows demonstrate the mechanism of target selection, not immunity to forgetting.

7. Learn from preferences as well as demonstrations

A demonstration supplies one desired answer. A preference example compares alternatives for the same request. For "a gentle comedy," a chosen answer might recommend an eligible catalog film with a grounded reason; a rejected answer might invent its runtime. Preference labels should reflect explicit criteria, and raters can disagree.

One common **reinforcement learning from human feedback (RLHF)** recipe trains a reward model from ranked responses, then optimizes the response-generating policy using that reward, often penalizing movement away from a reference policy. The policy is the model's distribution over possible responses. **Proximal Policy Optimization (PPO)** is one algorithm used in this stage. The [InstructGPT](#) paper demonstrates this pipeline after supervised tuning.

Direct Preference Optimization (DPO) uses chosen/rejected pairs directly, with an objective based on their relative likelihoods compared with a reference policy. It avoids separately fitting a reward model and running a PPO loop in that formulation. It is not simply SFT on the chosen text. See the [DPO](#) paper.

Both approaches can improve measured preferences without guaranteeing truth. A model may learn to sound convincing or exploit weaknesses in the reward signal. Chapter 28 explores alignment and preference learning; this chapter's workbook implements pretraining and SFT only.

▼ Go deeper: the comparison inside DPO

For prompt x , chosen reply y_w , and rejected reply y_l , a standard DPO loss uses:

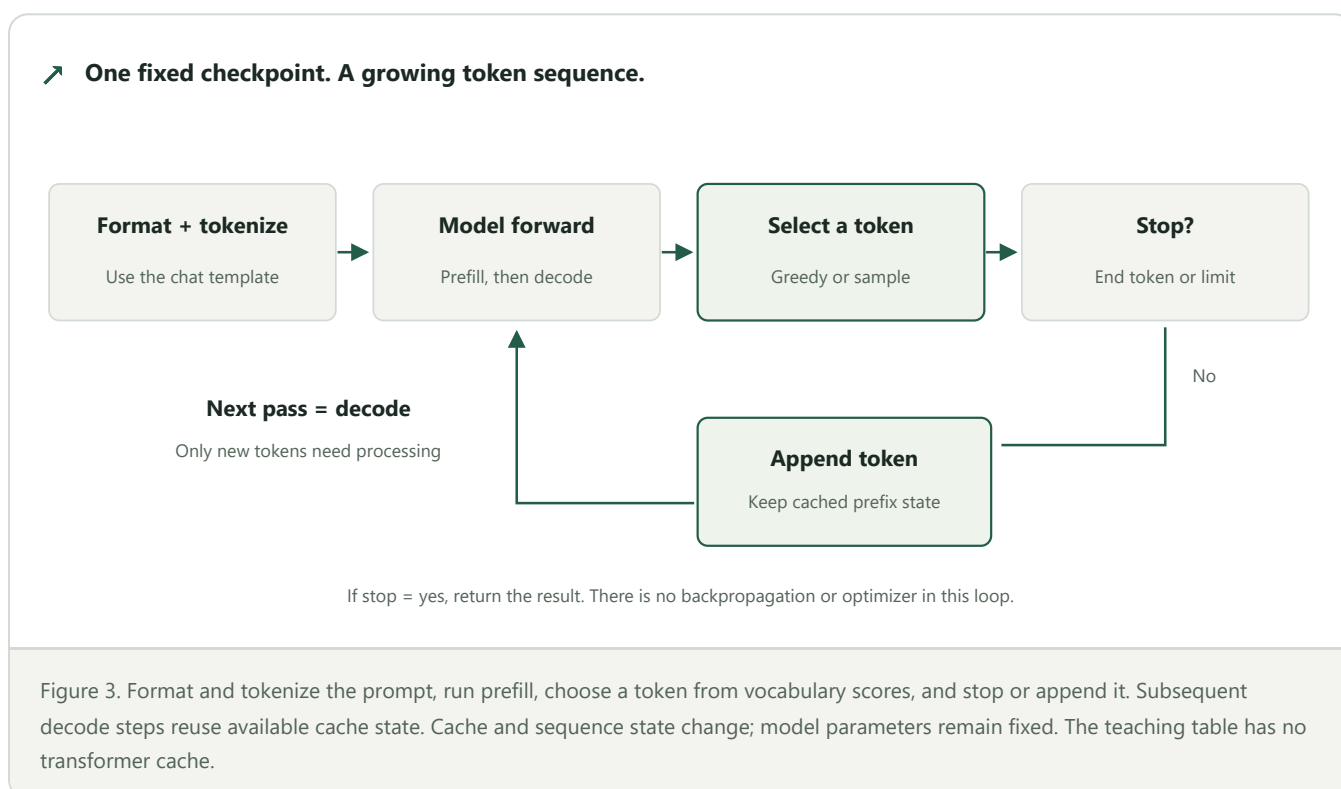
$$-\log \sigma \left(\beta \left[\log \frac{\pi_{\theta}(y_w|x)}{\pi_{ref}(y_w|x)} - \log \frac{\pi_{\theta}(y_l|x)}{\pi_{ref}(y_l|x)} \right] \right).$$

π_{θ} is the policy being trained, π_{ref} is a fixed reference, σ is the sigmoid, and β controls the objective's scale. Each reply log-probability sums its token log-probabilities. The objective favors an increased chosen-versus-rejected likelihood margin relative to the reference; it does not independently require every chosen reply's absolute probability to rise. Chapter 28 will work through preference optimization numerically.

8. Turn a conversation into model input

An application stores messages with roles such as user and assistant. A **chat template** converts them into the token sequence expected by a model, including role and turn delimiters. At generation time, the prefix may end with an assistant-start marker to indicate whose turn comes next. Real formats vary; our angle-bracket markers are only a teaching convention. Use the checkpoint's compatible tokenizer and template. [Hugging Face's chat-template guide](#) explains this conversion.

During **prefill**, a causal transformer processes the known prompt and produces the first next-token scores. During **decode**, each selected token extends the sequence and supports the next prediction. A KV cache can reuse earlier attention keys and values. The cache is temporary sequence state; it is not an optimizer update or a database of facts. See the [generation guide](#).



9. Choose tokens, stop, and evaluate the answer

Chapter 23 introduced decoding: greedy selection chooses the highest-probability token; sampling draws from a distribution, optionally transformed by temperature or filters. These choices affect generated text while leaving the checkpoint unchanged. A higher token probability measures model likelihood under a context, not factual confidence.

Generation ends when a configured end token, stop condition, or limit is reached. A maximum-new-token limit counts tokens, not words. Ending at that limit can leave an answer unfinished. Prompt content, template overhead, and generated tokens must fit the system's supported context budget; some systems also allocate budget to internal generated tokens.

Evaluate the complete movie assistant: Are the films real catalog entries? Do they satisfy the request? Are explanations supported? Does it handle “no match”? Also measure latency and cost. A lower token loss cannot replace these task checks. Use Chapters 15 and 19’s evaluation habits with separate development and final test sets.

▼ Go deeper: reproducibility and the training-generation gap

Teacher-forced evaluation conditions on the correct earlier tokens. Generation conditions on the model’s own earlier choices, so a mistaken token can change the context for everything that follows. Good held-out likelihood is useful evidence, but it does not guarantee a good multi-step answer.

Record checkpoint, tokenizer, template, prompt, decoding settings, and software versions when comparing runs. A fixed random seed controls a source of randomness; it does not promise identical results across all hardware, kernels, or service changes. Our small deterministic workbook specifies its vocabulary ordering, random-number generator, and tie-breaking rule so you can reproduce its traces locally.

10. Train, freeze, and inspect the result

Start with reply-only SFT and 80 updates. Compare the three checkpoints, then generate a comedy reply. Try zero SFT steps, switch to the prose prompt, and compare all-token SFT. Finally, set a one-token output limit to see why stopping is separate from knowing how to continue.

Same model. Two different jobs.

Train 15-token probability tables on movie text, then generate from a saved checkpoint. This is a two-token-context model, not a transformer or a useful recommender.

Default: 80 full-batch pretraining updates, then 80 reply-only SFT updates. The 15-token model conditions only on the previous two tokens. The learning rate is 3; each step averages the loss across the selected targets.

Checkpoint	Prose loss	Reply loss
Initial	2.7081	2.7081
After pretraining	0.3505	2.7081
After SFT	0.3505	0.0364

The reply-only mask excludes the three prompt targets and scores the four reply targets in each demonstration. Across four demonstrations, that is 16 targets per update; all-token training scores 28.

Frozen SFT checkpoint + comedy prompt + greedy selection → **Moonlight** → **Map** → . → **<eos>**. Visible reply: **Moonlight Map**. The first reply-token probability rises from 6.67% before SFT to 96.42% afterward. A one-token limit returns "Moonlight" and stops at the limit; a four-token limit reaches the end token.

With all-token SFT for 80 updates, prose loss rises to 1.1543 while reply loss falls to 0.0715. These are training-set measurements from independent context tables, not evidence about a real LLM's generalization.

This **trigram softmax table** has actual trainable logits but no embeddings, attention, or neural layers. Each prediction sees only the previous two tokens. Unseen contexts use a uniform distribution. It cannot generalize like a modern LLM; the tiny dataset is entirely visible and has no held-out split. Movie names are fictional.

The workbook contains `data.json`, `lifecycle.py`, `test_lifecycle.py`, and `README.md`. It uses Python 3.10+ and the standard library, without network calls or model downloads.

```
py lifecycle.py
py lifecycle.py --mask all --steps 80
py lifecycle.py --checkpoint pretrained --prompt prose
py lifecycle.py --steps 20 --method sample --temperature 1.5 --limit 8
py -m unittest -v test_lifecycle.py
```

Use `python` or `python3` if needed. The calculation command prints a JSON report containing training history, checkpoints, and generation. Invalid CLI choices exit with code 2. The browser displays rounded values; reports keep full precision.

Exercise 1: count the scored targets

There are four demonstrations, each with three prompt targets and four reply targets. How many targets enter each reply-only update? What changes with an all-token loss mask? Does the prompt disappear?

▼ Reveal the reasoning

Reply-only training scores $4 \times 4 = 16$ targets; all-token training scores $4 \times 7 = 28$. Both keep the prompt in context. The average loss and gradients use the number of selected targets as their denominator.

Exercise 2: diagnose a misleading metric

After all-token SFT, prose loss rises from 0.3505 to 1.1543 while reply loss falls to 0.0715. Has the model improved? Would reply-only training guarantee no forgetting in a transformer?

▼ Reveal the reasoning

It fits these conversation replies better and the earlier prose targets worse. "Improved" needs a task and evaluation set. The all-token mask changes a shared start-context row. Reply-only rows happen to be separate in this table; shared transformer parameters do not provide that protection. None of these training losses establishes unseen recommendation quality.

Exercise 3: trace a frozen answer

The default SFT checkpoint generates `Moonlight → Map → . → <eos>`. What is returned with a one-token limit? With a four-token limit? Which weights change if you switch to sampling?

▼ Reveal the reasoning

A one-token limit returns "Moonlight" and stops at the token limit. Four tokens reach the end token; the visible text is "Moonlight Map." The end token counts toward the limit but is removed from displayed text. Switching to sampling changes token selection, not weights. In this workbook the end-token condition takes precedence when both stopping conditions coincide.

11. Check your understanding

01 Which operation changes the model's learned parameters?

- A. An optimizer update using training gradients.
- B. Adding a retrieved movie description to the prompt.
- C. Reusing earlier attention keys and values.

Answer: A. Training updates parameters. Prompt content and a KV cache change runtime inputs or state. Ordinary inference does not run a training optimizer.

02 Reply-only SFT excludes user targets from direct loss. What happens to the user prompt?

- A. It must also be removed from attention.
- B. It remains context for predicting the assistant reply.
- C. Its words become the assistant's target labels.

Answer: B. A loss mask selects scored targets. It does not remove the prompt or replace the causal attention mask. In a neural model, reply losses can still backpropagate through representations of the prompt.

03 Which description correctly distinguishes SFT from preference optimization?

- A. SFT never changes weights, while preference optimization does.
- B. Both always train a separate reward model and run PPO.
- C. SFT learns from demonstration targets; preference optimization learns from comparisons or rewards.

Answer: C. Both can update parameters, using different supervision. DPO can use chosen/rejected pairs directly without separately fitting a reward model and running PPO.

04 A frozen checkpoint generates an answer. What happens during ordinary autoregressive decoding?

- A. The correct complete answer is supplied as a teacher-forced target.
- B. A token is selected, added to the prefix, and used to predict the next token.
- C. The optimizer updates weights after each generated word.

Answer: B. The growing sequence and any cache change, while the checkpoint weights stay fixed. Generation ends at a configured stopping condition or limit.

05 The workbook's reply loss falls to 0.0364. What does that establish?

- A. The model can recommend any unseen film accurately.
- B. The next recommendation has a 96.36% chance of being factually correct.
- C. The checkpoint assigns higher likelihood to these visible demonstration targets.

Answer: C. This is mean token loss on training examples. It is neither task accuracy nor calibrated factual confidence; generalization requires separate evaluation.

12. Connect learning to efficient systems

You can now follow the full chain: **prepare data** → **predict targets** → **update parameters** → **adapt behavior** → **evaluate a checkpoint** → **format a prompt** → **generate with fixed weights**. Separate what the training objective rewards from what the application must achieve.

Chapter 26: Efficient and advanced LLM systems will examine how memory, computation, and deployment choices affect the cost of running these models. Keep this chapter’s boundary in mind: weights, temporary activations, and cached sequence state serve different purposes.

▼ Quick-reference glossary

Term	Meaning
Pretraining / continued pretraining	Initial broad training / further training on a text objective
SFT / instruction tuning	Learning from demonstrated target replies / adapting to follow instructions
Teacher forcing	Predict using the observed earlier tokens during training
Loss mask / attention mask	Select scored targets / control visible input positions
Batch / step / epoch	Group for computation / optimizer update / pass through a dataset
Checkpoint	Saved model state; resumable training may require additional state
RLHF / reward model	Learning from human feedback / model scoring candidate behavior
DPO / reference policy	Direct preference objective / comparison distribution used in optimization
Chat template	Serialization of messages into the model’s expected token format
Prefill / decode / KV cache	Process a prompt / extend it / reuse earlier attention state
Catastrophic forgetting	Adaptation degrades an earlier learned capability

Sources and further reading

Primary references are linked beside the concepts: Chinchilla, InstructGPT, Direct Preference Optimization, and Hugging Face’s SFT, chat-template, and generation documentation. The movie examples, diagrams, independent-row training model, and worked calculations are original teaching material.